



Better Estimated Time of Arrival using Deep Learning-based Traffic Prediction



Scott Shen · [Follow](#)

Published in AI4SM

16 min read · Oct 26, 2023



Listen



Share

By Bocheng Zhang, Chaoyue Gong, Chang Shen as part of course project of ECE1724H: Bio-inspired Algorithms for Smart Mobility. Dr. Alaa Khamis, University of Toronto, 2023.

Abstract

In this article, the team introduced a novel autoregressive deep-learning model to predict traffic speed. The growing problem of urban traffic congestion has prompted people to crave and explore new solutions in order to find the best answer. The article analyzes the root cause of inaccurate estimation of arrival time, and then proposes a novel model that deals with both spatial and temporal information. This model can be incorporated into a practical navigation software. The model is evaluated on the historical traffic data in San Jose. Based on the evaluation, the model can predict traffic speed in the future, and thus provide a more accurate arrival time. The model performance matches the state of art solutions such as STEP and GMAN [1] [2].

1. Introduction

With the increasing urban population and the widespread use of transportation, urban traffic congestion has obviously become a globally significant problem that cannot be ignored. Due to the widespread issue of inaccurate estimated arrival times in current map software, it has had a considerable impact on society. Commuters and those who need to arrive on time heavily rely on accurate predictions of arrival times from map software. Therefore, the team began a deep exploration of solutions for traffic prediction to provide residents with more reliable arrival times.

After a period of exploration, the team found that one of the challenges in traffic prediction is the exceptionally complex factors that cause changes in urban traffic conditions. These factors include school areas during drop-off and pick-up times, hazardous zones, specific holidays, and more. Each of these factors can have a certain impact on current traffic conditions. After discussions within the team, it was decided to not consider these factors and instead directly use data on vehicle speeds at specific target points at different times each day for traffic prediction. Therefore, the team utilized the PEMS-BAY dataset collected by the California Transportation Agencies Performance Measurement System (PeMS), which gathers information on vehicle speeds at specific points on various highways, divided into different time periods.

The purpose of this article is to find a suitable deep learning method to predict estimated time of arrival (ETA) accurately. The team finds the Autoregressive Model could be the answer demanded [3]. The rest of the article will demonstrate the pros and cons of the model by comparing different papers in performance and mathematical models.

2. Literature Review

Traffic flow prediction is an important part of the Intelligent Transport System (ITS) [7]. Also, there are lots of challenges that need to be overcome to achieve the traffic forecasting, like traffic patterns, queuing patterns and time. Throughout history, a variety of methods have been employed for traffic prediction, for example, Graph Convolutional Network, Diffusion Convolutional and Recurrent Neural Network (DCRNN), Spatial-Temporal Graph Networks, etc.

2.1 Graph Neural Networks (GNNs)

In recent years, Graph Neural Networks (GNNs) are widely applied in traffic predicates. It is a type of neural network designed to operate directly on graphs, which are data structures made up of nodes (vertices) and edges. According to Oliver and Luis, by expanding the definition of “proximity,” Graph Neural Networks broaden the learning bias imposed by Convolutional Neural Networks and Recurrent Neural Networks [10]. This creates the feature to have arbitrary complex connections to manage traffic not only ahead or behind, but also along adjacent and intersecting roads. However, most traffic prediction methods using GNNs, like Graph Convolutional Networks (GCNs) and Graph Attention Networks (GANs), focus on spatial dependencies and ignore information on temporal dependencies [9]. In

recent research, GCN based approaches have been widely used in traffic prediction. Chenhen Zhang and his teammate mentions that hybrid GCN-based model that depicts the temporal dependency with LSTM and the spatial dependency with random walks on the traffic network [4] [9]. Additionally, Spatio-Temporal Graph Convolutional Networks (ST-GCN) are convolutional networks that use both spatial and temporal convolutional elements.

2.2 Spatial-Temporal Graph Networks

ST- GCNs have the ability to capture the spatial and temporal correlation in graph structured data. Yan et al 's model for action recognition showed how well ST-GCNs handle spatial-temporal data [8]. In these graphs, nodes represent entities (such as road segments or intersections), and edges show spatial interactions between those elements. The temporal aspect is then captured when node features change over time. Diffusion convolution is used in a model developed by Li et al to reflect the dynamic spatial-temporal interplay in traffic networks [8]. The advantages of ST-GCNs were highlighted in their strategy, particularly their improved predictability and ability to model congestion propagation.

According to Xiangyuan Kong and his colleagues, Spatial-Temporal Graph Attention Network (STGAT) is an end-to-end deep learning based dual path framework and it can be directly generalized to the graph with arbitrary structure [6]. By stacking gated temporal convolution layers, STGAT can handle lengthy temporal sequences. ST-GAN can capture potential spatial dependencies and use more useful information for prediction. Compared with other models, ST-GAN applies the attention concentration mechanism to the spatiotemporal framework, so this model can be applied to more graphs. Furthermore, experimental results on two public real-world transportation network datasets, METR-LA and PEMS-BAY, show that Xiangyuan Kong and his colleagues' STGAT surpasses state-of-the-art baselines. And this model can effectively transfer between graphs of different structures [6].

2.3 Innovation and Use of Predecessor Methods

GAT does have a higher number of uses in traffic prediction. However, the road map dataset (PeMS) is not perfect. The map only shows the highway traffic information, and does not consider the non-highway part of the road. According to the GAT mathematical model, it only considers the data of sensors which are adjacent to the target sensor. It is hard to add the potential relationship in which two sensors are

not connected in the “highway map” but there is a non-highway road connected in the real map. The extra computational steps have to be added to use the GAT method. Thus, the GCN is used in the GNN process of the Autoregressive Model. GCN’s mathematical model using a self-adjusted matrix to describe the relationship among nodes. GCN could have higher performance when learning from a partially observable environment.

3. Problem Formulation and Modeling

3.1 Traffic Prediction Problem

We are given a traffic network described by a weighted undirected graph $G=(V, E, w)$ and the traffic speed $s(e, t)$ at time t on edge e , where the weight function $w: E \rightarrow R$ describes the physical road length of each edge. Traffic prediction is to predict the traffic speed in the future based on historical traffic speed data, that is to predict $s(e, t)$, $t > t_0$ at time point t_0 based on $s(e, t)$, $t \leq t_0$.

3.2 Time of Arrival

Given a path including n nodes on the traffic network $(v_1, v_2, \dots, v_n)$, where $v_i \in V$, $(v_i, v_{i+1}) \in E$, $\forall i$, the time of arrival $A(v_1, v_2, \dots, v_n; t)$ if we departure at time t is axiomatically defined by the following three rules:

(A1) a path containing only one node has a time of arrival of 0,

$$A(v_i; t) = 0, \forall i, t$$

(A2) time of arrival of an edge is the road length divided by its traffic speed,

$$A(v_i, v_{i+1}; t) = \frac{w(v_i, v_{i+1})}{s((v_i, v_{i+1}), t)}, \forall i, t$$

(A3) time of arrival of a path is the time spent on the first edge plus the time spent on the rest of the path,

$$A(v_i, v_{i+1}, \dots, v_n; t) = A(v_i, v_{i+1}; t) + A(v_{i+1}, \dots, v_n; t + A(v_i, v_{i+1}; t)), \forall n > 2, i, t$$

We can further expand the recursive form to make it simpler,

$$A(v_1, v_2, \dots, v_n; t) = \sum_{i=1}^n T_i$$

where T_i is the time spent on edge (v_i, v_{i+1}) , which is recursively defined as

$$T_i = \frac{w(v_i, v_{i+1})}{s((v_i, v_{i+1}), t + \sum_{j=1}^{i-1} T_j)}$$

3.3 Estimated Time of Arrival

Since we only have historical traffic speed data $(s(e, t), t \leq t_0)$, it's natural to make an estimation of the time of arrival only based on current traffic speed on each edge, that is

$$\hat{T}_i = \frac{w(v_i, v_{i+1})}{s((v_i, v_{i+1}), t_0)}$$

However, the traffic speed of one edge may change when we arrive at that edge in the future. Imagine a long and straight road $v_1 - v_2 - v_3$, where each edge has a physical road length of 10 km . At the time we depart t_0 , there is a large volume of vehicles on edge (v_1, v_2) , so the traffic of that edge is slow, say $s((v_1, v_2), t_0) = 50 \text{ km/h}$. And (v_2, v_3) is fairly empty with an average traffic speed $s((v_2, v_3), t_0)$ of 100 km/h . Current solution may give an estimated time of arrival by simply adding current estimated time $10/50 + 10/100 = 0.3$ hours. But since a large number of vehicles is moving from (v_1, v_2) to (v_2, v_3) , it's very likely that when we start driving on (v_2, v_3) , other vehicles are also on the second edge, causing a much lower traffic speed. We may end up also driving at 50 km/h on (v_2, v_3) . And the actual time of arrival is $10/50 + 10/50 = 0.4$ hours.

Traffic prediction can help reduce this error in calculating ETA. If we can make a good prediction of future traffic speed in the near future, denoted by $\hat{s}(e, t)$, $t > t_0$, then we can presumably calculate a better ETA by predicting $\hat{T}_i$ based on $\hat{s}$

$$\hat{T}_i = \frac{w(v_i, v_{i+1})}{\hat{s}((v_i, v_{i+1}), t_0 + \sum_{j=1}^{i-1} \hat{T}_j)}$$

Back to the example above, the model can be trained to be aware of the moving traffic on the straight road and predict $\hat{s}((v_1, v_2), t_0) = 50 \text{ km/h}$, $\hat{s}((v_2, v_3), t_0 + T_1) = 50 \text{ km/h}$, and give an estimation $A((v_1, v_2, v_3); t_0) = 0.4 \text{ hours}$. The key component of this method is how to model $\hat{s}$ accurately.

4. Proposed Solution

4.1 Architecture

Our team proposed a novel autoregressive neural network model to predict traffic [3]. Autoregressive models iteratively call a single procedure (named cell) to encode or decode data. In our proposed model, an AR cell has two inputs: data at one time point and a state from the previous result. It also has two outputs: predicted data and an updated state.

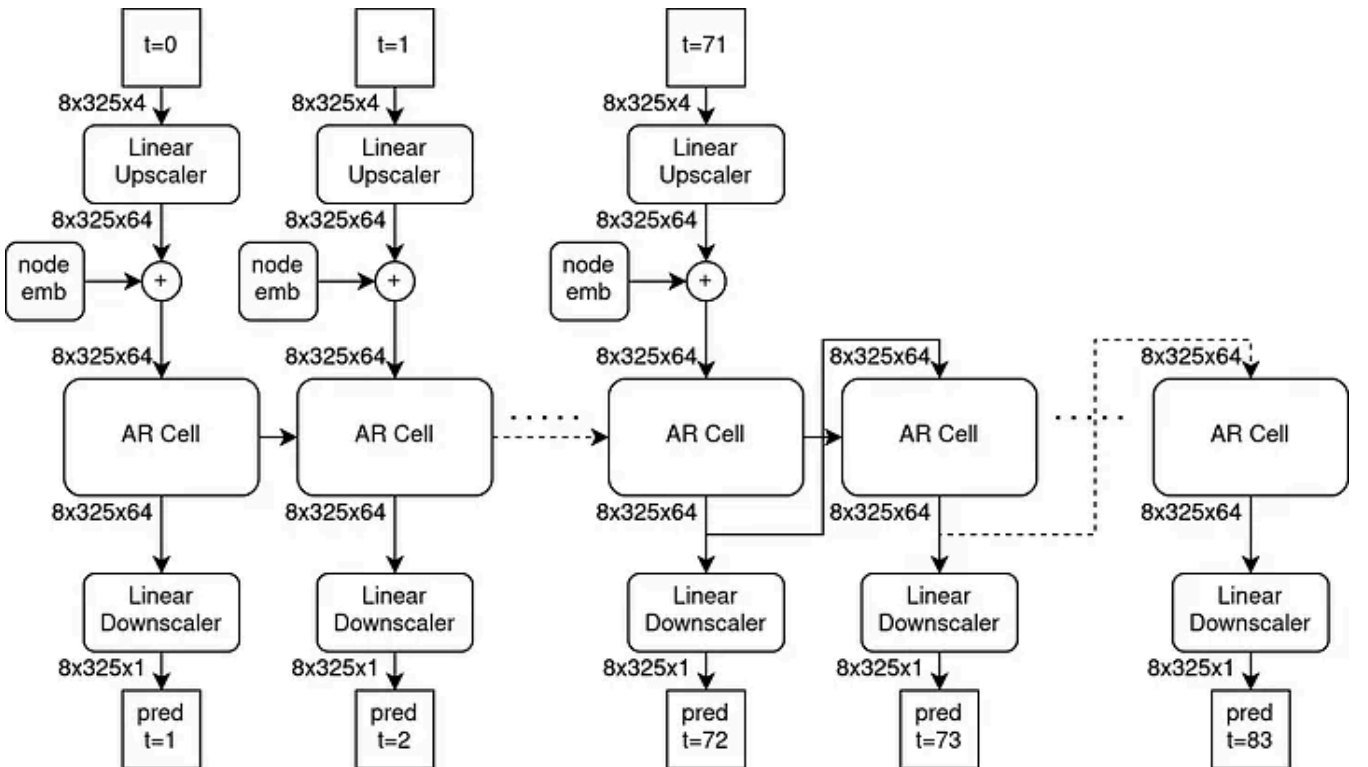


Fig 1.1: Autoregressive Neural Network Model

The execution of the model can be separated into two phases: encoding and decoding. Both phases use the AR cell but in different ways.

In encoding, the input data at each time point is upscaled into dimensionality of the model (which is 64 in our implementation) and fed to the AR cell. The AR cell

generates the prediction of the next time point. The prediction is then downscaled to output size (which is 1 representing the traffic speed). The state is passed to the next call of the AR cell. In decoding, where the model doesn't have access to the ground truth, the AR cell accepts predictions from previous results as input instead, and outputs predictions as usual. Again, the prediction is then downscaled to output size.

This design of AR model enables the user to input sequences of varying length, and predict sequences of any desired length. Of course, cumulative errors will impact the accuracy of predicting excessively long sequences.

The AR Cell is iteratively called in the execution. It includes two Long Short-Term Memory (LSTM) layers, one GCN layer, and one linear layer [4]. The input data is first processed through the first LSTM layer along its time dimension, and then passed through the GCN layer. The first LSTM is responsible for extracting intra-node temporal information, and GCN is used to extract spatial information. The output is then adjusted by a linear feed-forward layer, and then passed to another LSTM layer to capture inter-node temporal dependencies in the sequence.

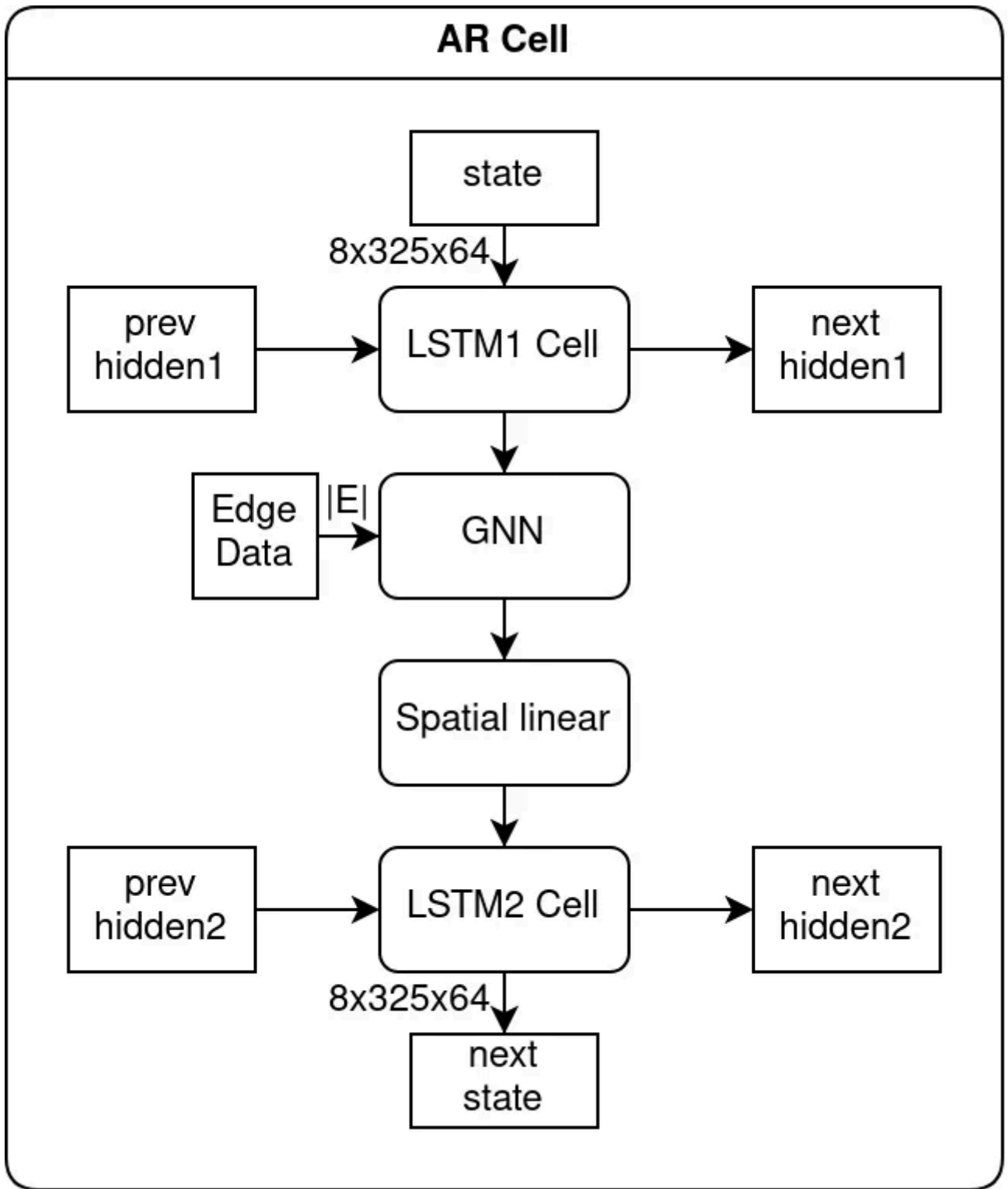


Fig 1.2: AR cell Structure

4.2 Preprocessing and node embedding

The dataset includes two time series: traffic flow and traffic speed. The traffic flow is scaled by a factor of 500 using linear normalization. Traffic speed is scaled by a factor of 100. This makes sure the raw data is ranged from 0 to 1. In order to add time in a day to the input data, the team used a 2-dimensional positional encoding [5]. By using 2d positional encoding, two features, namely cosine and sine of time in

a day, are added to the input feature. So input feature size is 4 for each time point for each node.

Another problem is that the first LSTM is executed solely on the temporal dimension, and there isn't any information it can make use of to distinguish data between nodes. To alleviate this issue, we add a learned node embedding to the hidden state.

4.3 Training and dataset

The team combines two different training methods: teacher-forcing training and unrolling training. In teacher forcing training, the model is fed with the correct output at each time step during training. Teacher forcing training will boost the model performance in short term prediction. However, due to the training restriction, it is not good at holding long term information. In the unrolling training, the model generates its own predictions at each time step and uses them as inputs for the subsequent time steps, rather than relying on the true outputs from the training data. Our train procedure uses teacher-forcing for one iteration, and unrolling training for another. This combined strategy gives us the benefits of both methods.

The model is trained on the historical traffic data in San Jose from January 1, 2022 to June 30, 2022. The data set contains 181 days with 325 sensors located in different points in the highway. Each sensor will record the traffic flow (number of vehicles passed in 5 mins) and the vehicle speed for every 5 minutes. Therefore, the total number of data is $181 \times (24 \times 60 / 5) \times 325 \times 2$. In the implementation, the team uses a batch size of 8 and try to predict traffic speed of next hour at the granularity of 5 mins using traffic flow and speed data of the last 6 hours. So the input size is $8 \times 325 \times 72 \times 4$ and the output size is $8 \times 325 \times 12 \times 1$.

5. Dataset

The PEMS-BAY dataset will be used in this project, which traffic data are collected by California Transportation Agencies Performance Measurement System (PeMS) [8]. The data set collects the information about 3991 sensors and records 18 information for each sensor, like direct, latitude, longitude, and length, related to the San Francisco Bay Area. Each sensor will be treated as a traffic node. We will separate the dataset into a training set, testing set and validation set. For the training set, which contains 70% of data, used to train the model. For the testing set, which contains 20% of data, used to test the model. For the validation set, which

contains 10% of data, used to fine-tune the model’s hyperparameters and assess how well the model generalizes to new data during the training phase.

Exploratory Spatial Data Analysis (ESDA)

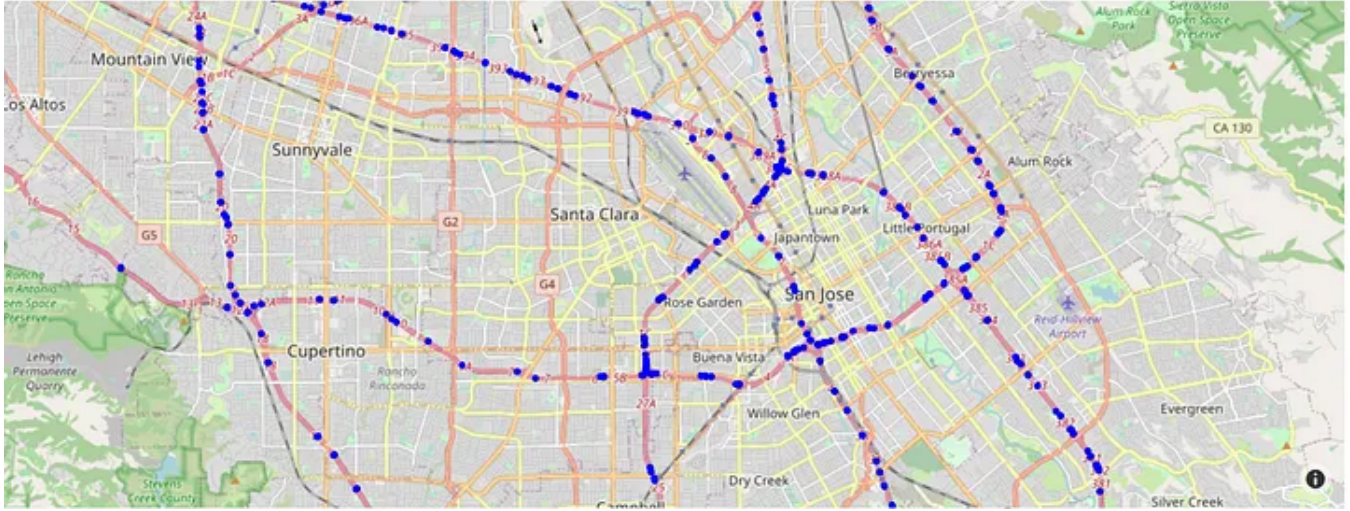


Figure 2: Sensor distribution of PEMS-BAY dataset

Figure 2, shows the data about each sensor and plots all of them on a map. From the map we know that all sensors are located on the highway and most of them are two sensors together and responsible for different directions. For example:

ID	Fwy	Dir	District	County	City
401857	580	W	4	1	53000
401858	580	E	4	1	53000
State_PM	Abs_PM	Latitude	Longitude	Length	Type
37.09	53.156	37.763173	-122.149746	2.515	ML
37.09	53.004	37.763133	-122.149977	2.515	ML
Lanes	Name	User_ID_1	User_ID_2	User_ID_3	User_ID_4
4	E OF FONTAIN ST OC	DT666	L4-W-33-580-03709	101542	422932
4	E OF FONTAIN ST OC	DT666	L4-E-33-580-03709	101543	420336

Table 1: Dataset Sample

In the dataset, the sensor with id 401857 and 401858, they are together but facing two directions, 401857 (west), 401858 (east). The figure 2.1(a) shows those two sensor's location in the map.



Figure 2.1 (a): The image about sensor 401875 and 401858

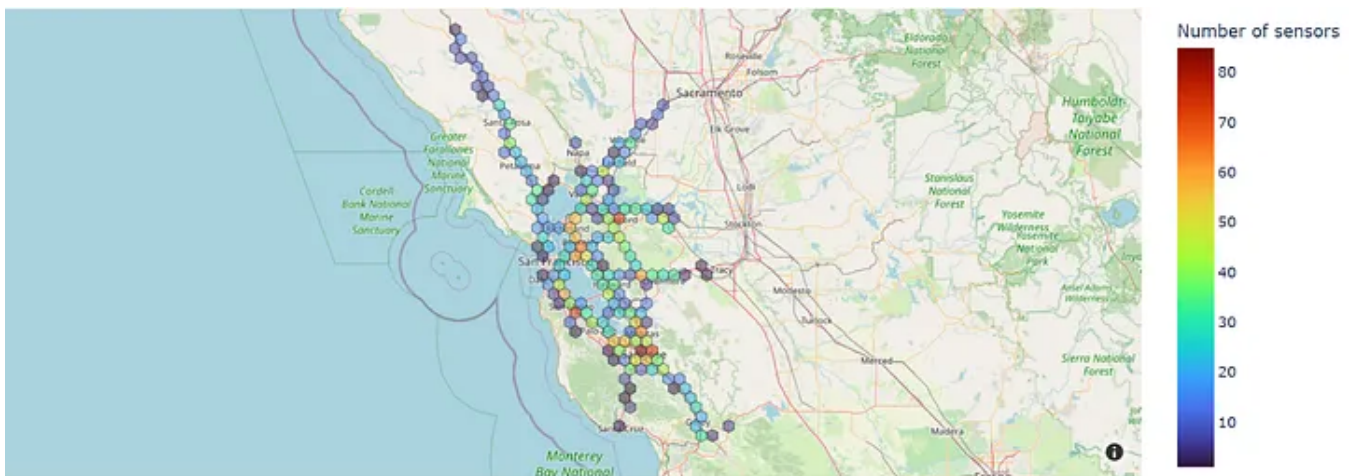


Figure 3: The density of sensor distribution of PEMS-BAY dataset

Figure 3 showcases the distribution of sensors across a significant portion of the California region, notably covering areas from Monterey Bay up to the vicinity of Sacramento. The color-coded dots represent the number of sensors in each location, with the provided legend indicating that the density varies from 1 to 80 sensors. The high density area of the sensor is in San Jose, because of the warmer colors on the legend.

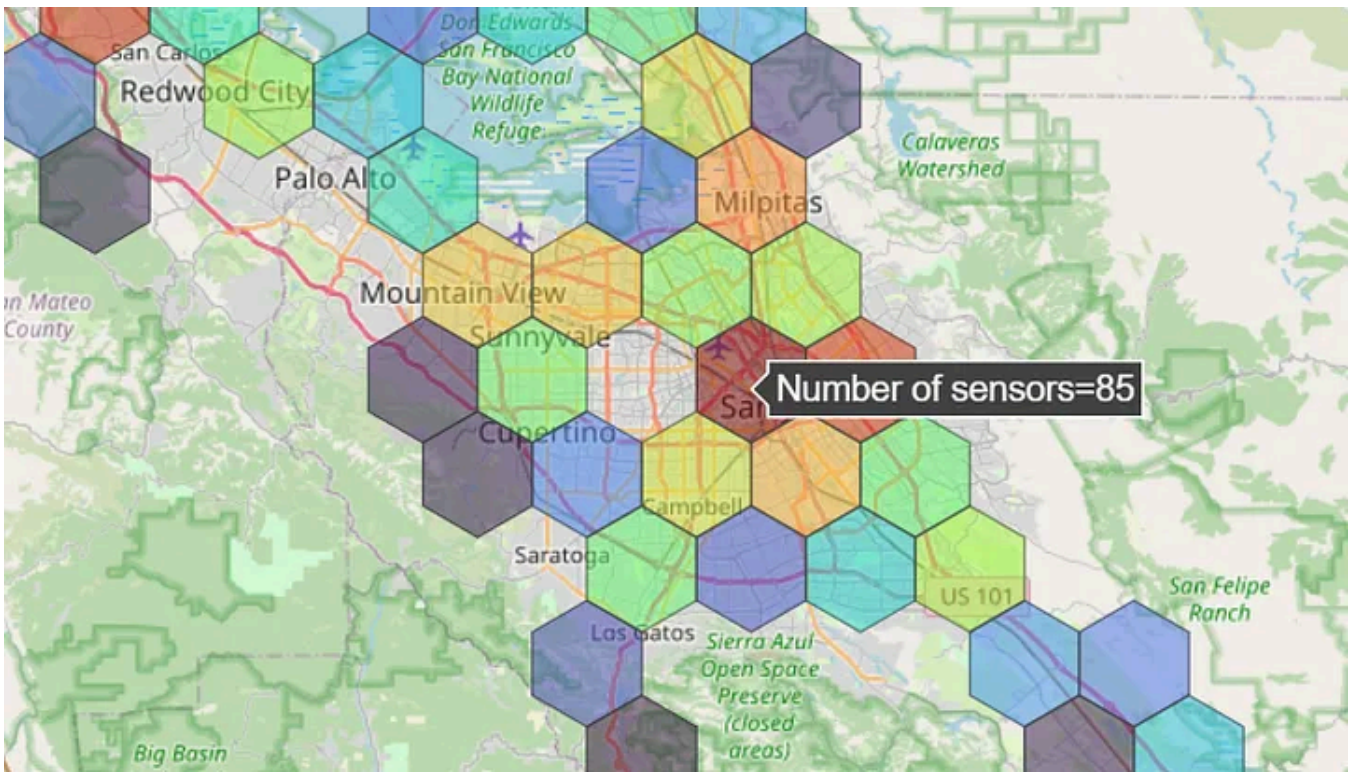


Figure 3.1(a): The high density area with the max number of sensors = 85

6. Performance Evaluation

The team used historical traffic data in San Jose from June 1, 2022 to June 30, 2022 to evaluate the model performance.

In order to compare with state-of-art traffic prediction models such as STEP and GMAN, we do a 3-Step prediction [1] [2]. That is, given historical data, predict the traffic data in the next 12 minutes at the granularity of 5 minutes, which is 3 steps.

The proposed model has a mean absolute error (MAE) of 1.31, and a rooted mean squared error of 2.65. Compared with that, STEP has a MAE of 1.26 and a RMSE of 2.63 and Graph WaveNet has a MAE of 1.31 and RMSE of 2.74.

Model	Mean Absolute Error (MAE)	Rooted Mean Squared Error (RMSE)
STEP	1.26	2.63
GMAN	1.34	2.91
Proposed Model	1.31	2.65

Table 2: MAE and RMSE Comparing

Next, we plot the predicted traffic speed of selected days. However, cumulative errors will impact the accuracy of predicting excessively long sequences, and 24 hours (that is 288 steps) is too overwhelming for current models. Instead, we feed 6 hour data to the model and predict the traffic speed for the first hour. This is repeated 24 times to get predictions for each hour. To present the results, we concatenate each 1 hour prediction, with each slice distinguished by vertical gray bars. As shown in Fig 4, the model can predict future traffic speed relatively accurately.

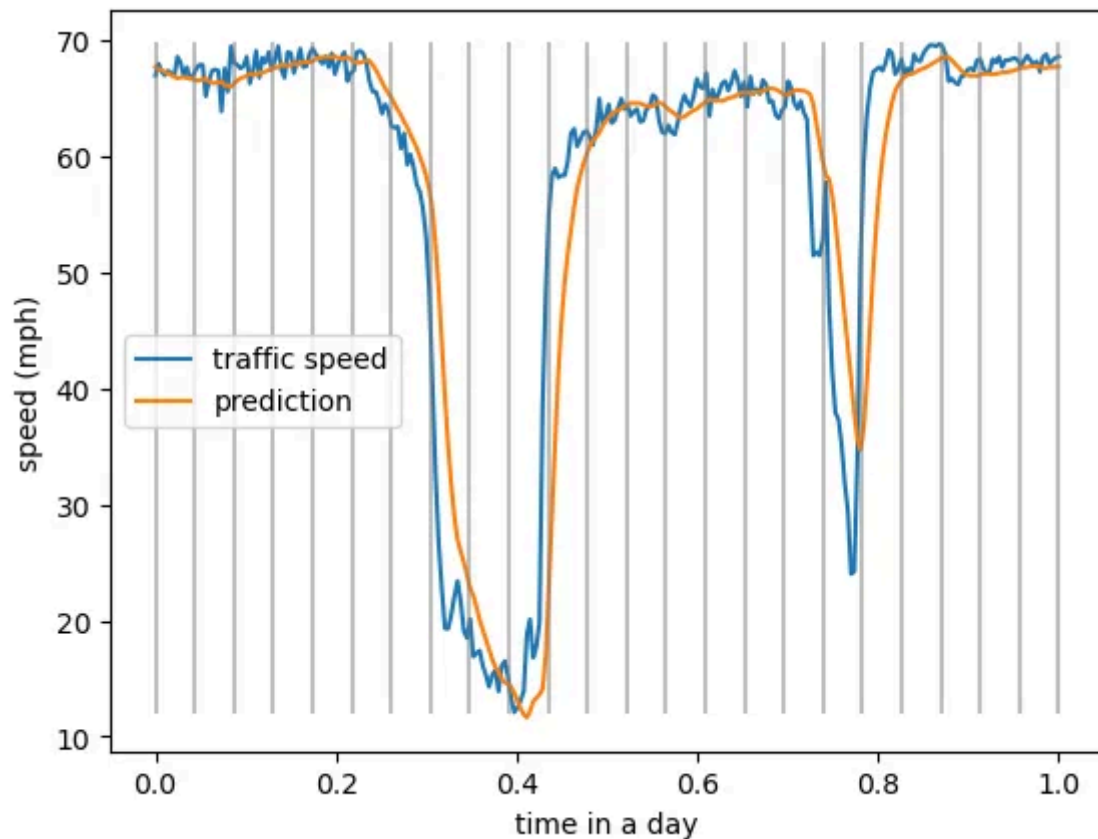


Figure 4: Prediction for Node 250 Throughout the Entire Day on June 20th

However, the model isn't perfect. In the prediction for node 73 on June 14th, the model overestimates the traffic speed at 7am to 9am by 20 mph. This can cause a huge error for users using this road.

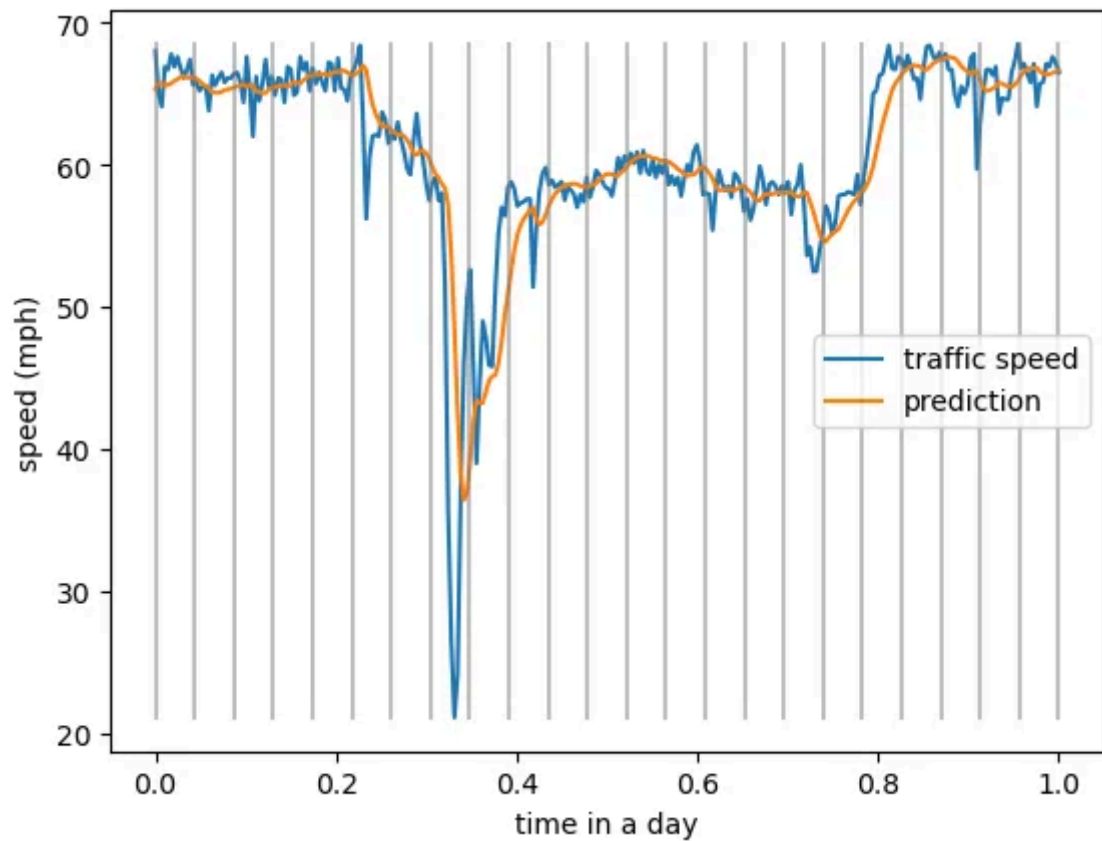


Figure 4: Prediction for Node 73 Throughout the Entire Day on June 14th

The team also builds a simple navigation program based on the deep-learning model.

Fig 5 shows the result of generating two paths from node 195 to 212 on 17:30 June 25th.

Path 1 contains nodes [195, 304, 307, 308, 252, 113, 91, 77, 58, 99, 311, 141, 324, 321, 233, 169, 230, 167, 212]. According to the model's prediction, ETA for this path is 20.69 minutes, whereas the actual time is 20.46 minutes.

Path 2 contains nodes [195, 303, 292, 288, 287, 27, 225, 189, 247, 103, 223, 221, 109, 219, 186, 246, 36, 26, 217, 19, 18, 215, 63, 24, 175, 176, 212]. The ETA is 26.12 minutes, whereas the actual time is 25.73 minutes.



Figure 5: Generated Path from Node 195 to 212

Since the team used a deep-learning solution, it doesn't have too many parameters to tune from. Changing the dimensionality of the model from 64 to 128 doesn't improve model performance much, while reducing it to 32 also doesn't change the performance.

However, one measurable parameter is the choice of time encoding. In the proposed solution, positional encoding is used. But if not, say the time in a day is represented by a single parameter ranging from 0 to 1, where 0 represents 12:00 am and 1 represents 23:59 pm. As shown in Fig 6, the model underestimates at the beginning of the day by almost 20mph. And this error even persists for 5 slices.

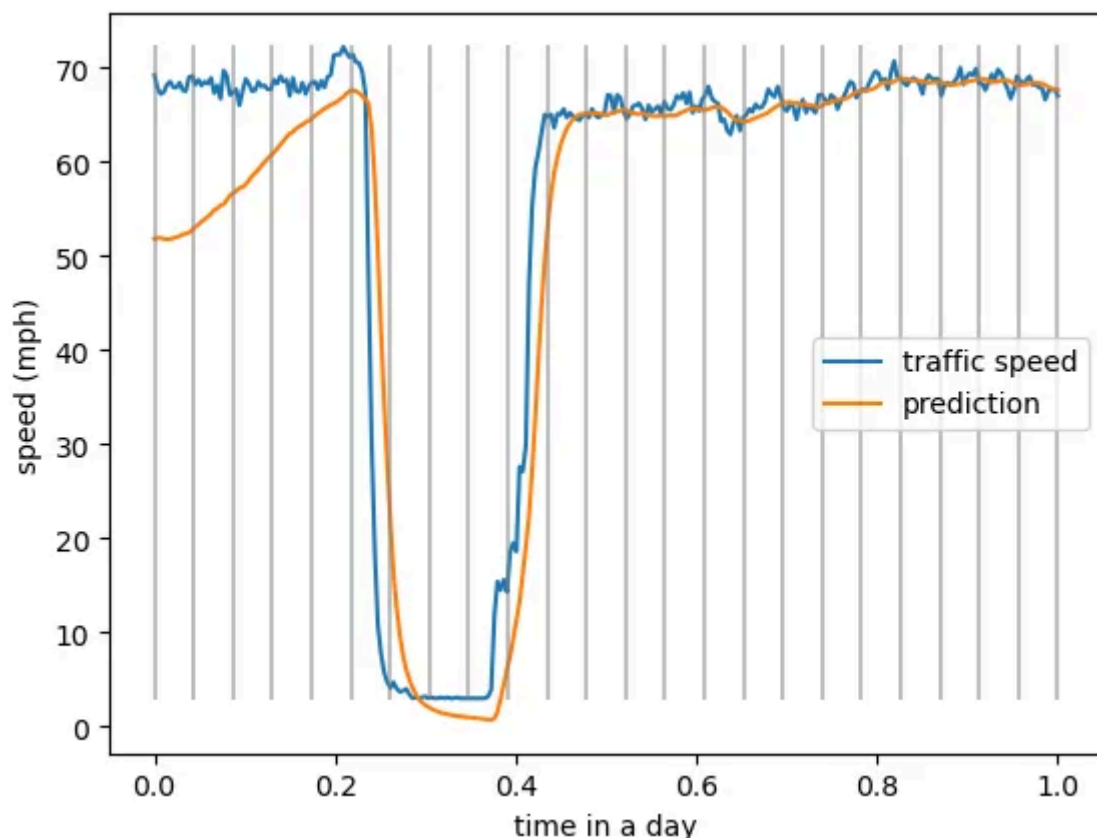


Figure 6: Prediction for Node 25 Throughout the Entire Day on June 20th, w/o Positional Encoding

This phenomena can be explained as the sudden change in simple time encoding. As shown in Fig 7, when one day ends, the time value changes from 0.9 to 1.1, the simple time encoding changes suddenly. This will result in huge state change in the model, and cause a huge prediction error. On the other hand, the positional encoding we used doesn't have this problem. The time feature will change smoothly from 0.9 to 1.1.

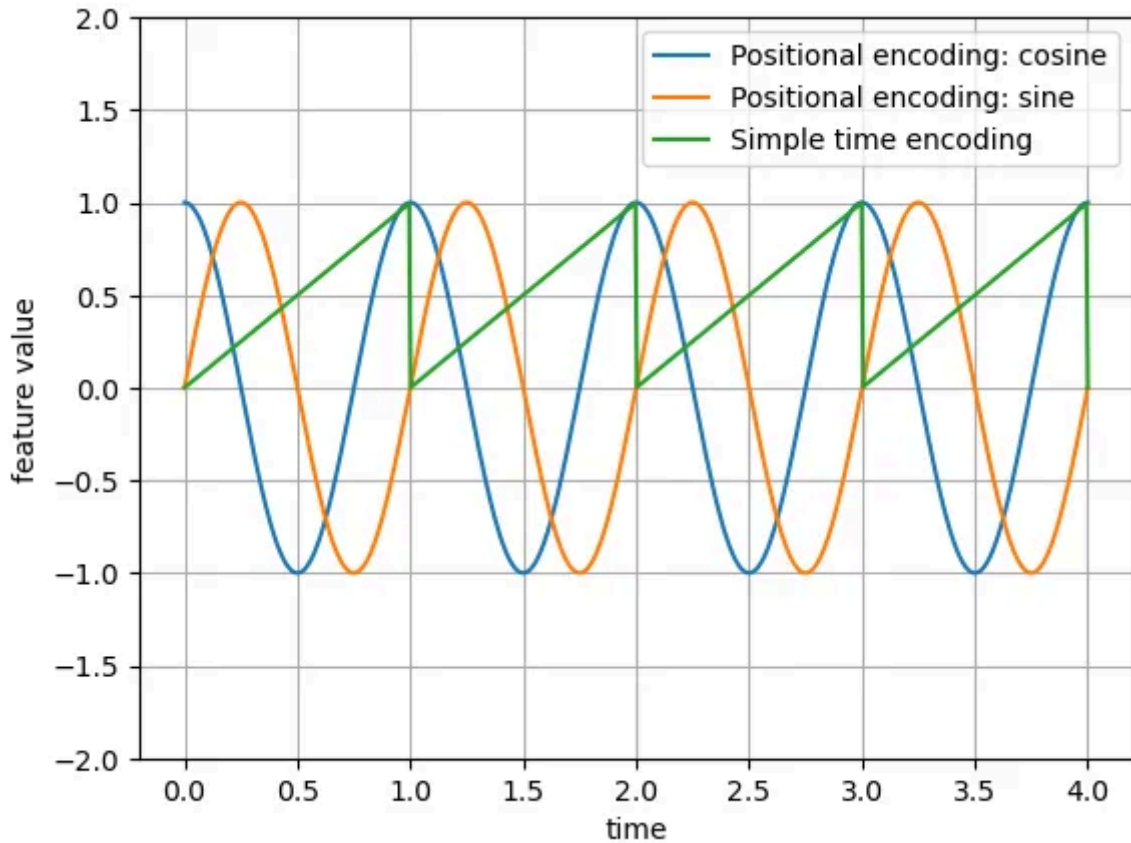


Figure 7: Time Embedding Comparison: Positional Encoding and Simple Time Encoding

7. Conclusions & Recommendations

In conclusion, the team aimed at enhancing traffic prediction and the accuracy of estimated times of arrival (ETAs) has yielded promising results. By leveraging the introduced autoregressive model, our team provides a reliable solution of estimating arrival time. The result is analyzed based on the PEMS-BAY dataset and focused on highway traffic, combining Long Short-Term Memory (LSTM) and Graph Convolutional Network (GCN) within a predictive model. The findings indicate that while traditional models focus primarily on spatial dependencies, the team's model efficiently incorporates both spatial and temporal elements, leading to more accurate traffic predictions. However, there are some limitations in applying this model to real-world road maps. Some complex road sections can create a large

number of edges in the graph, causing ineffectiveness in model prediction. This research contributes to the field of traffic prediction by offering a more reliable and efficient method for estimating travel times. As urban traffic continues to be a critical issue, the advancement of such predictive technologies is vital.

In the future, the work could explore integrating non-highway traffic data and refining the model to adapt to a broader range of traffic conditions. For example, it can be used for route planning in cities, helping people to reduce their driving time in urban areas by planning their routes appropriately. Also, the spatial and temporal layers of the model can be refined as multiple unified layers. In addition, more conditions can be taken into account when making predictions, such as weather, emergencies, and so on.

8. Colab Notebook

Google Colaboratory Edit description colab.research.google.com	
---	--

Reference:

- [1] Shao, Zezhi, et al. "Pre-training enhanced spatial-temporal graph neural network for multivariate time series forecasting." *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2022.
- [2] Zheng, Chuanpan, et al. "GMAN: A graph multi-attention network for traffic prediction." *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. №01. 2020..
- [3] Graves, Alex. "Generating sequences with recurrent neural networks." arXiv preprint arXiv:1308.0850 (2013).
- [4] Sak, Haşim, Andrew Senior, and Françoise Beaufays. "Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition." arXiv preprint arXiv:1402.1128 (2014).

- [5] Gehring, Jonas, et al. “Convolutional sequence to sequence learning.” International conference on machine learning. PMLR, 2017.
- [6] X. Kong, W. Xing, X. Wei, P. Bao, J. Zhang, and W. Lu, “STGAT: Spatial-temporal graph attention networks for traffic flow forecasting,” IEEE Access, vol. 8, pp. 134 363–134 372, 2020.
- [7] A. A. Kashyap, S. Raviraj, A. Devarakonda, S. R. N. K, S. K. V, and S. J. Bhat, “Traffic flow prediction models — a review of deep learning techniques,” Cogent Engineering, vol. 9, no. 1, dec 2021.
- [8] Y. Li, R. Yu, C. Shahabi, and Y. Liu, “Diffusion convolutional recurrent neural network: Data-driven traffic forecasting,” 2018.
- [9] C. Zhang, J. J. Q. Yu, and Y. Liu, “Spatial-temporal graph attention networks: A deep learning approach for traffic forecasting,” IEEE Access, vol. 7, pp. 166 246–166 256, 2019.
- [10] A. Derrow-Pinion, J. She, D. Wong, O. Lange, T. Hester, L. Perez, M. Nunkesser, S. Lee, X. Guo, B. Wiltshire, P. W. Battaglia, V. Gupta, A. Li, Z. Xu, A. Sanchez-Gonzalez, Y. Li, and P. Velickovic, “Eta prediction with graph neural networks in google maps,” in Proceedings of the 30th ACM International Conference on Information and Knowledge Management, ser. CIKM ’21. ACM, Oct. 2021. [Online]. Available: <http://dx.doi.org/10.1145/3459637.3481916>



Written by Scott Shen

2 Followers · Writer for AI4SM

Follow

